

hotchpotch / **static-embedding-japanese**   like 32



Sentence Similarity



sentence-transformers



ONNX



Safetensors



8 datasets



Japanese



English

loss:MatryoshkaLoss

loss:MultipleNegativesRankingLoss

arxiv:2205.13147



License: mit



Train

Deploy

Use this model



Model card

Files

xet



Community

Downloads last month

-

Downloads are not tracked for this model. [How to track](#) 

🔧 Inference Providers NEW

🔧 Sentence Similarity

This model isn't deployed by any Inference Provider.



Ask for provider support

📁 Datasets used to train hotchpotch/static-embedding-japanese

以下の文章は、記事 [100倍速で実用的な文章ベクトルを作れる、日本語 StaticEmbedding モデルを公開](#) からの転載です。

🔗 static-embedding-japanese

文章の密ベクトルは、情報検索・文章判別・類似文章抽出など、さまざまな用途に使うことができます。しかしながら最先端のTransformerモデルは小さいモデルでも、とりわけCPU環境では処理速度が遅いため実用でないこともしばしばあります。

この課題を解決する新しいアプローチとして、先日公開されたTransformerモデル「ではない」 [StaticEmbeddingモデル](#)は、例えば [intfloat/multilingual-e5-small](#) (以下mE5-small)とのベンチマーク比較では85%のスコアという最低十分な性能で、何よりCPUで動作時に126倍高速に文ベクトルを作成することができる、という驚きの速度です。

というわけで、早速日本語(と英語)で学習させたモデル `sentence-embedding-japanese` を作成し、公開しました。

- <https://huggingface.co/hotchpotch/static-embedding-japanese>

日本語の文章ベクトルの性能を評価する JMTEB の結果は以下です。総合スコアでは `mE5-small` には若干及ばないまでも、タスクによっては勝っていたりしますし、他の日本語baseサイズ bertモデルよりもスコアが高いこともあるぐらい、最低限実用できそうな性能が出ていますね。本当にそんなに性能が出るのか実際に学習させてみるまでは半信半疑でしたが、驚きです。

Model	Avg(micro)	Retrieval	STS	Classification	Reranking	Clustering	PairClassification
text-embedding-3-small	69.18	66.39	79.46	73.06	92.92	51.06	62.27
multilingual-e5-small	67.71	67.27	80.07	67.62	93.03	46.91	62.19
static-embedding-japanese	67.17	67.92	80.16	67.96	91.87	40.39	62.37

なお、StaticEmbedding 日本語モデル学習などの技術的なことは記事の後半に書いているので、興味がある方はどうぞ。

🔗 利用方法

利用は簡単、SentenceTransformer を使っていつもの方法で文章ベクトルを作れます。今回は GPU を使わず、CPU で実行してみましょう。なお SentenceTransformer は 3.3.1 で試しています。

```
pip install "sentence-transformers>=3.3.1"
```

```

from sentence_transformers import SentenceTransformer

model_name = "hotchpotch/static-embedding-japanese"
model = SentenceTransformer(model_name, device="cpu")

query = "美味しいラーメン屋に行きたい"
docs = [
    "素敵なカフェが近所にあるよ。落ち着いた雰囲気です。ゆっくりできるし、窓際の席からは公園の景色も見え。",
    "新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕も素晴らしい。",
    "あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。",
    "おすすめの中華そばの店を教えてください。とりわけチャーシューが手作りで柔らかくてジューシーなものがいいです。"
]

embeddings = model.encode([query] + docs)
print(embeddings.shape)
similarities = model.similarity(embeddings[0], embeddings[1:])
for i, similarity in enumerate(similarities[0].tolist()):
    print(f"{similarity:.04f}: {docs[i]}")

```

(5, 1024)

0.1040: 素敵なカフェが近所にあるよ。落ち着いた雰囲気です。ゆっくりできるし、窓際の席からは公園の景色も見え。

0.2521: 新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕も素晴らしい。

0.4835: あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。

0.3199: おすすめの中華そばの店を教えてください。とりわけチャーシューが手作りで柔らかくてジューシーなものがいいです。

このように、queryにマッチする文章のスコアが高くなるように計算できてますね。この例文では、例えばBM25ではqueryに含まれる「ラーメン」のような直接的な単語が文章に出ていないため、うまくマッチさせることが難しいでしょう。

続いて、類似文章タスクの例です。

```

sentences = [
    "明日の午後から雨が降るみたいです。",
    "来週の日曜日は天気が良いそうだ。",
    "あしたの昼過ぎから傘が必要になりそう。",

```

```

    "週末は晴れるという予報が出ています。",
]

embeddings = model.encode(sentences)
similarities = model.similarity(embeddings, embeddings)

print(similarities)

# 一つ目の文章と、その他の文章の類似度を表示
for i, similarity in enumerate(similarities[0].tolist()):
    print(f"{similarity:.04f}: {sentences[i]}")

```

```

tensor([[1.0000, 0.2814, 0.3620, 0.2818],
        [0.2814, 1.0000, 0.2007, 0.5372],
        [0.3620, 0.2007, 1.0000, 0.1299],
        [0.2818, 0.5372, 0.1299, 1.0000]])
1.0000: 明日の午後から雨が降るみたいです。
0.2814: 来週の日曜日は天気が良いそうだ。
0.3620: あしたの昼過ぎから傘が必要になりそう。
0.2818: 週末は晴れるという予報が出ています。

```

こちら、類似文章が高スコアになる結果になりました。

またTransformerモデルを利用してCPUで文章ベクトルを作った場合、少ない文章量でもだいぶ時間がかか、という経験をされた方も多いと思います。StaticEmbedding モデルではCPUがそこそこ速ければ一瞬で終わるはず。さすが100倍速。

🔗 出力次元を小さくする

標準で作られる文ベクトルの次元は1024ですが、これをさらに小さく次元削減することもできます。例えば128を指定してみましょう。

```

# truncate_dim は 32, 64, 128, 256, 512, 1024 から指定
model = SentenceTransformer(model_name, device="cpu", truncate_dim=128)

query = "美味しいラーメン屋に行きたい"
docs = [

```

```
"素敵なカフェが近所にあるよ。落ち着いた雰囲気です。ゆっくりできるし、窓際の席からは公園の景色も見  
"新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕も  
"あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。",  
"おすすめ中華そばの店を教えてください。とりわけチャーシューが手作りで柔らかくてジューシーなん
```

```
embeddings = model.encode([query] + docs)  
print(embeddings.shape)  
similarities = model.similarity(embeddings[0], embeddings[1:])  
for i, similarity in enumerate(similarities[0].tolist()):  
    print(f"{similarity:.04f}: {docs[i]}")
```

```
(5, 128)
```

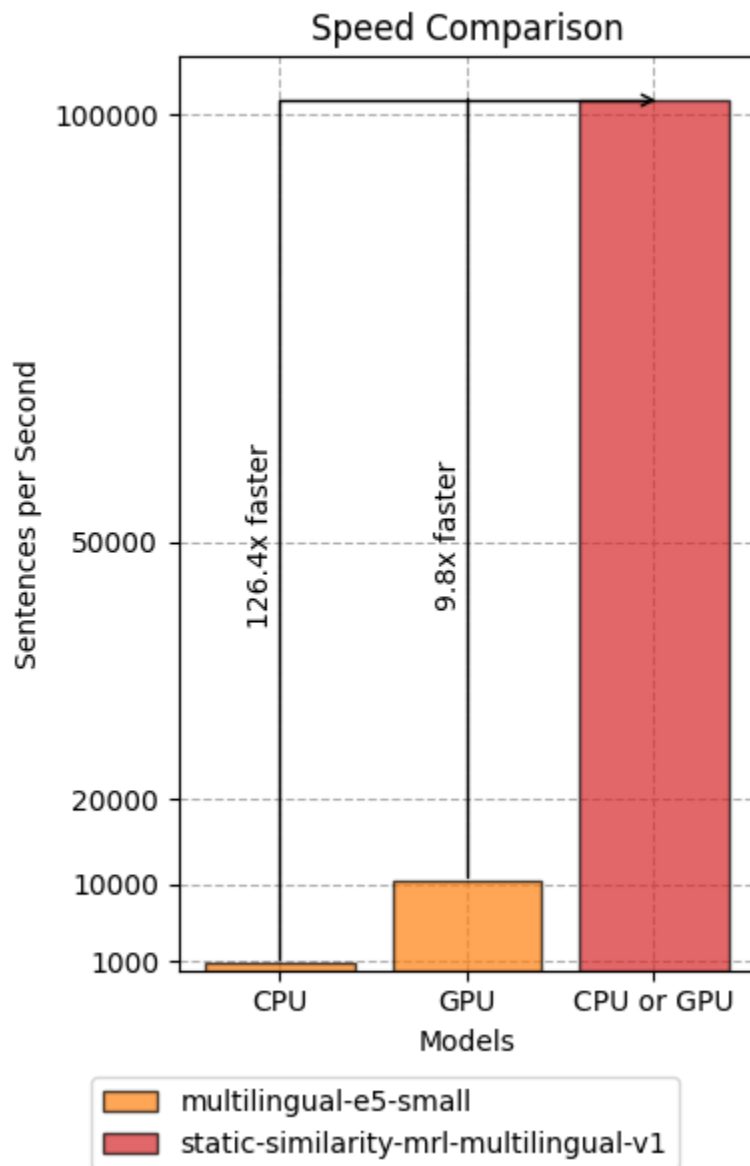
```
0.1464: 素敵なカフェが近所にあるよ。落ち着いた雰囲気です。ゆっくりできるし、窓際の席からは公園の景色  
0.3094: 新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕  
0.5923: あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。  
0.3405: おすすめ中華そばの店を教えてください。とりわけチャーシューが手作りで柔らかくてジューシーな
```

128次元のベクトルになり、結果のスコアも若干変わりましたね。次元が小さくなったことで、性能が少々劣化しています(後半にベンチマークを記載)。ただ1024次元から128次元に減ることで、保存するストレージサイズが減ったり、検索時などに利用する類似度計算コストが約8倍速になったりとなったりと、用途によっては小さい次元の方が嬉しいことも多いでしょう。

🔗 なぜCPUで推論が高速なの？

StaticEmbedding はTransformerモデルではありません。つまりTransformerの特徴である "Attention Is All You Need" なアテンションの計算が一切ないのです。文章に出てくる単語トークンを1024次元のテーブルに保存して、文ベクトル作成時にはその平均をとっているだけです。なお、アテンションがないので、文脈の理解などはしていません。

また内部実装では PyTorch の nn.EmbeddingBag を使って、全てを連結したトークンとオフセットを渡して処理することで、PyTorch の最適化で高速なCPU並列処理とメモリアクセスがされているようです。



元記事の速度評価結果によるとCPUではmE5-smallと比べて126倍速らしいですね。

🔗 評価結果

JMTEBでの全ての評価結果は[こちらJSONファイルに記載](#)しています。[JMTEB Leaderboard](#)で他のモデルと見比べると、相対的な差がわかるでしょう。JMTEBの全体の評価結果はモデルサイズを考えると、すこぶる良好です。なお、JMTEBのmr-tidyタスクは700万文章のベクトル化を行うので処理に時間がかかりかかる(モデルにもよりますがRTX4090で1~4時間ほど)と思います。これもStaticEmbeddingsでは非常に速く、RTX4090では約4分で処理終わることができました。

🔗 情報検索でBM25の置き換えができそうか?

JMTEBの中の情報検索タスクの[Retrievalの結果](#)を見てみましょう。StaticEmbeddingではmr-tidyの項目が著しく悪いですね。mr-tidyは他のタスクに比べて文章量が圧倒的に多く(700万文章)、つまり所大量の文章を検索するようなタスクでは結果が悪い可能性がありそうです。文脈を無視した単純なトークンの平均なので、増えれば増えるほど似た平均の文章が出てくるとすると、そういう結果にもなり得そうですね。

ので、大量の文章の場合、BM25よりもだいぶ性能が悪い可能性がありそうです。ただ、少ない文章で、ずばりの単語マッチが少ない場合は、BM25よりも良好な結果になることが多そうですね。

なお情報検索タスクのjaqketの結果が他のモデルに対してやたら良いのは、jaqketの問題を含むJQaRa (dev, unused)を学習しているからといっても、高すぎる感じで謎です。testの情報リークはしていないと思うのですが...

🔗 クラスタリング結果が悪い

こちらも詳細は追っかけていませんが、スコア的には他のモデルよりもだいぶ悪い結果ですね。クラス分類タスクは悪くないので不思議です。埋め込み空間がマトリョーシカ表現学習で作られた影響もあるのでしょうか。

🔗 JQaRA, JaCWIR でのリランキングタスク評価

[JQaRA](#)の結果はこちら。

model_names	ndcg@10	mrr@10
static-embedding-japanese	0.4704	0.6814
bm25	0.458	0.702
multilingual-e5-small	0.4917	0.7291

[JaCWIR](#)の結果はこちら。

model_names	map@10	hits@10
<u>static-embedding-japanese</u>	0.7642	0.9266
bm25	0.8408	0.9528
<u>multilingual-e5-small</u>	0.869	0.97

JQaRa 評価は BM25 よりは若干良く、mE5-small よりは若干低い、JaCWIR は BM25, mE5 よりだいぶ低い感じの結果になりました。

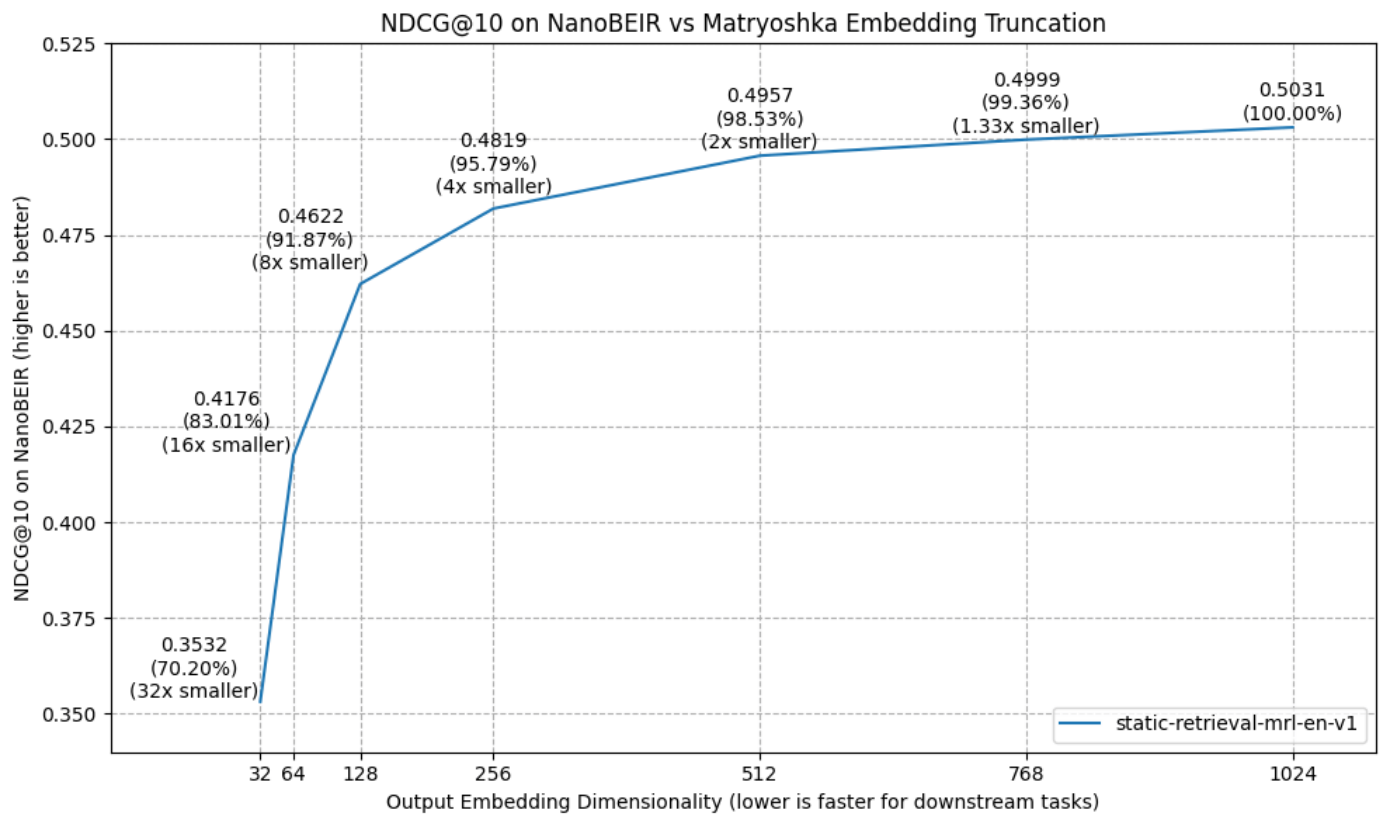
JaCWIR は query から探しあてる文章が、Web 文章のタイトルと概要文なので、いわゆる「綺麗な」文章ではないケースも多いです。transformer モデルはノイズに強いので、単純なトークン平均の StaticEmbedding ではスコアに差がつけられるのも納得ですね。BM25 は特徴的な単語が出現した文章にマッチするので、JaCWIR でもノイズとなるような文章上の単語はクエリにそもそもマッチしないため、Transformer モデルと競争力のある結構良い結果を残しています。

この結果から、StaticEmbedding は Transformer / BM25 に比べ、ノイズを多く含む文章の場合はスコアが悪い可能性があります。

🔗 出力次元の削減

StaticEmbedding で出力される次元は、学習次第ですが今回作成したものは 1024 次元とそこそこのサイズです。次元数が大きいと、推論後のタスク(クラスタリングや情報検索など)に計算コストがかかってしまいます。しかしながら、学習時にマトリョーシカ表現学習(Matryoshka Representation Learning(MRL))をしているため、1024 次元をさらに小さな次元へと簡単に次元削減ができます。

MRL は、学習時に先頭のベクトルほど重要な次元を持ってくることで、例えば 1024 次元でも先頭の 32, 64, 128, 256... 次元だけを使って後ろを切り捨てるだけで、ある程度良好な結果を示しています。



このグラフ参照元の[StaticEmbeddingの記事](#)によると、128次元で91.87%、256次元で95.79%、512次元で98.53%の性能を維持しているようです。精度にそこまでシビアではないが、その後の計算コストを下げたい場合、ガッと次元削減して使う、という用途にも使えそうですね。

🔗 StaticEmbedding 日本語モデルでの次元削減結果

JMTEB では、出力時にモデルのパラメータを制御できるため、truncate_dim オプションを渡すことで、次元削減した結果のベンチマークも簡単に計測できます。素晴らしいですね。というわけで、StaticEmbedding 日本語モデルでも、次元削減した結果でベンチマークをとってみました。

次元数	Avg(micro)	スコア割合(%)	Retrieval	STS	Classification	Reranking	Clustering	PairClassification
1024	67.17	100.00	67.92	80.16	67.96	91.87	40.39	62.37
512	66.57	99.10	67.63	80.11	65.66	91.54	41.25	62.37
256	65.94	98.17	66.99	79.93	63.53	91.73	42.55	62.37

次元数	Avg(micro)	スコア割合(%)	Retrieval	STS	Classification	Reranking	Clustering	PairClassification
128	64.25	95.65	64.87	79.56	60.52	91.62	41.81	62.33
64	61.79	91.98	61.15	78.34	58.23	91.50	39.11	62.35
32	57.93	86.24	53.35	76.51	55.95	91.15	38.20	62.37

スコアの変化を見ると、512次元へと次元削減した場合はやたらRetrieval, Classification, Reranking の性能が悪くなります。むしろ256次元まで次元削減してしまった方が良好な結果に。256次元では、スコア的には次元削減する前のモデルの98.93%なのですが、これはクラスタリングの結果がなぜか1024次元よりも良くなってしまったためですね。

512次元でのスコア計測が間違っていたので修正しました。マトリョーシカ表現学習がうまく反映され、次元数を削ると若干のスコア低下が見られますが、次元数が減ったためその後のコストが抑えられそうですね。

クラスタリングタスクにおいては128次元まで次元削減しても1024次元よりもスコアが高い、という本来情報量を削らない方がスコアが良くなりそうなのに、クラスタリングタスクのみは逆にスコアが上がってしまう興味深い結果となりました…。マトリョーシカ表現学習では、先頭の次元の方が全体的な特徴を踏まえているので、クラスタリング用途には(クラスタリングのアルゴリズムにもよりますが)、特徴的な前の方の次元のみで後ろの次元を使わない方が良質な結果が得られる、ということなのかもしれません。

というわけで、static-embedding-japanese モデルで次元削減する時は、512,256,128次元あたりが性能と次元削減のバランスが取れてそうですね。

🔗 StaticEmbedding モデルを作ってみて

正直、単純なトークンのembeddingsの平均でそんなに性能出るとは半信半疑だったのですが、実際に学習させてみてシンプルなアーキテクチャなのに性能の高さにびっくりしました。Transformer 全盛のこの時代に、古き良き単語埋め込みの活用モデルで、実世界で利活用できそうなモデルの出現に驚きを隠せません。

CPUでの推論速度が速い文ベクトル作成モデルは、ローカルCPU環境で大量の文章の変換などはもとより、エッジデバイスだったりネットワークが遅い(リモートの推論サーバを叩けない)環境だったり、色々と活用できそうですね。

🔗 StaticEmbedding 日本語モデル学習のテクニカルノート

🔗 なぜうまく学習できるのか

StaticEmbedding は非常にシンプルで、文章をトークナイズしたIDで単語の埋め込みベクトルが格納されているEmbeddingBagテーブルからN次元(今回は1024次元)のベクトルを取得し、その平均を取るだけです。

これまで、単語埋め込みベクトルといえば、word2vec や GloVe のように Skip-gram や CBOW を用いて単語の周辺を学習してきました。しかし、StaticEmbedding では文章全体を用いて学習しています。また、対照学習を使って大量の様々な文章を巨大バッチで学習しており、良い単語の埋め込み表現の学習に成功しています。

対照学習は、基本的に正例以外全てを負例として学習するため、例えばバッチサイズ2048なら1の正例に対して2047の負例を2048通り、つまり2048x2047で約400万の比較を学習します。そのため、元の単語空間に対して適切な重みを更新しながら、学習を進めることができます。

🔗 学習データセット

日本語モデル学習にあたり、対照学習で利用できるデータセットとして、以下を作成し使用しました。

- [hotchpotch/sentence_transformer_japanese](#)
 - SentenceTransformerで学習しやすいカラム名と構造に整えたものです。
 - `(anchor, positive), (anchor, positive, negative), (anchor, positive, negative_1, ..., negative_n)` といった構造になっています。

- 以下のデータセットを基に hotchpotch/sentence_transformer_japanese を作成しました。毎度ながらデータセットの作者の方々・とりわけ hpprc 氏に感謝です。
 - <https://huggingface.co/datasets/hpprc/emb>
 - https://huggingface.co/datasets/hotchpotch/hpprc_emb-scores のリランカースコアを使用し、positive(≥ 0.7) / negative(≤ 0.3) のフィルタリングを行いました。
 - <https://huggingface.co/datasets/hpprc/llmjp-kaken>
 - <https://huggingface.co/datasets/hpprc/msmarco-ja>
 - <https://huggingface.co/datasets/hotchpotch/msmarco-ja-hard-negatives> のリランカースコアを用いて、positive(≥ 0.7) / negative(≤ 0.3) のフィルタリングを行いました。
 - <https://huggingface.co/datasets/hpprc/mqa-ja>
 - <https://huggingface.co/datasets/hpprc/llmjp-warp-html>
- 上記の作成したデータセットの中で、以下を使用しました。なお、情報検索を強化したかったため、情報検索に適したデータセットのデータはオーギュメンテーションで件数を多めに学習させています。
 - httprc_auto-wiki-nli-triplet
 - httprc_auto-wiki-qa
 - httprc_auto-wiki-qa-nemotron
 - httprc_auto-wiki-qa-pair
 - httprc_baobab-wiki-retrieval
 - httprc_janli-triplet
 - httprc_jaquad
 - httprc_jqara
 - httprc_jsnli-triplet
 - httprc_jsquad
 - httprc_miracl

- [httprc_mkqa](#)
- [httprc_mkqa-triplet](#)
- [httprc_mr-tydi](#)
- [httprc_nu-mnli-triplet](#)
- [httprc_nu-snli-triplet](#)
- [httprc_quiz-no-mori](#)
- [httprc_quiz-works](#)
- [httprc_snow-triplet](#)
- [httprc_llmjp-kaken](#)
- [httprc_llmjp_warp_html](#)
- [httprc_mqa_ja](#)
- [httprc_msmarco_ja](#)
- 英語データセットには、以下のデータセットを利用しています。
 - [sentence-transformers/msmarco-co-condenser-margin-mse-sym-mnrl-mean-v1](#)
 - [sentence-transformers/squad](#)
 - [sentence-transformers/all-nli](#)
 - [sentence-transformers/trivia-qa](#)
 - [nthakur/swim-ir-monolingual](#)
 - [sentence-transformers/miracl](#)
 - [sentence-transformers/mr-tydi](#)

🔗 日本語トークナイザ

StaticEmbedding を学習するためには、HuggingFace のトークナイザライブラリの `tokenizer.json` 形式で処理可能なトークナイザを使うと簡単そうだったので、[hotchpotch/xlm-roberta-japanese-tokenizer](#) というトークナイザを作成しました。語彙数は 32,768 です。

このトークナイザは、~~wikipedia 日本語、wikipedia 英語(サンプリング)、cc-100(日本語, サンプルサンダ)~~(訂正:作成コードを確認したところ、wikipedia日本語のみを利用していました)のデー

タを unidic で分割し、sentencepiece unigram で学習したものです。XLM-Roberta 形式の日本語 トークナイザとしても機能します。今回はこのトークナイザを利用しました。

🔗 ハイパーパラメータ

大元の学習コードとの変更点やメモは以下の通りです。

- batch_size を大元の 2048 から 6072 に設定しました。
 - 対照学習で巨大なバッチを処理するとき、同一バッチ内にポジティブとネガティブが含まれると学習に悪影響を与える可能性があります。これを防ぐために `BatchSamplers.NO_DUPLICATES` オプションがあります。しかし、バッチサイズが巨大だと同一バッチに含めないためのサンプリング処理に時間がかかることがあります。
 - 今回は `BatchSamplers.NO_DUPLICATES` を指定し、RTX4090 の 24GB に収まる 6072 に設定しました。バッチサイズはさらに大きい方が結果が良い可能性があります。
- epoch数を1から2に変更しました
 - 1よりも2の方が良い結果になりました。ただし、データサイズがもっと大きければ、1の方が良い可能性があります。
- スケジューラ
 - 標準のlinearから、経験則でより良いと感じるcosineに変更しました。
- オプティマイザ
 - 標準のAdamW のままです。adafactorに変更した場合、収束が悪くなりました。
- learning_rate
 - 2e-1 のままです。値が巨大すぎるのではないかと疑問に思いましたが、低くすると結果が悪化しました。
- dataloader_prefetch_factor=4
- dataloader_num_workers=15
 - トークナイズとバッチサンプラのサンプリングに時間がかかるため、大きめに設定しました。

🔗 学習リソース

- CPU
 - Ryzen9 7950X
- GPU
 - RTX4090
- memory
 - 64GB

このマシンリソースで、フルスクラッチ学習にかかった時間は約4時間でした。GPUのコア負荷は非常に小さく、他のtransformerモデルでは学習時に90%前後で張り付くのに対して、StaticEmbeddingではほとんど0%でした。これは、巨大なバッチをGPUメモリに転送する時間が大半を占めているためかと思われます。そのため、GPUメモリの帯域幅が速くなれば、学習速度がさらに向上する可能性があります。

🔗 さらに性能向上へ

今回利用したトークナイザはStaticEmbedding向けに特化したものではないため、より適したトークナイザを使用すれば性能が向上する可能性があります。バッチサイズをさらに巨大化することで、学習の安定性が向上し、性能向上が見込めるかもしれません。

また、さまざまなドメインや合成データセットを利用するなど、より幅広い文章リソースを学習に組み込むことで、さらなる性能向上が期待できます。

🔗 大元の学習コード

学習に使用したコードは、以下で MIT ライセンスで公開しています。スクリプトを実行すれば再現できる、はず...!

- <https://huggingface.co/hotchpotch/static-embedding-japanese/blob/main/trainer.py>

🔗 ライセンス

static-embedding-japanese はモデル重み・学習コードを MIT ライセンスで公開しています。

A Day in the Life

100倍速で実用的な文章ベクトルを作れる、日本語 StaticEmbedding モデルを公開

2025年01月20日 21時00分

文章の密ベクトルは、情報検索・文章判別・類似文章抽出など、さまざまな用途に使うことができます。しかしながら最先端のTransformerモデルは小さいモデルでも、とりわけCPU環境では処理速度が遅いため実用でないこともしばしばあります。

この課題を解決する新しいアプローチとして、先日公開されたTransformerモデル「ではない」 [StaticEmbedding モデル](#)は、例えば [intfloat/multilingual-e5-small](#) (以下mE5-small)とのベンチマーク比較では85%のスコアという最低十分な性能で、何よりCPUで動作時に126倍高速に文ベクトルを作成することができる、という驚きの速度です。

というわけで、早速日本語(と英語)で学習させたモデル `sentence-embedding-japanese` を作成し、公開しました。

- <https://huggingface.co/hotchpotch/static-embedding-japanese>

日本語の文章ベクトルの性能を評価する [JMTEB](#) の結果は以下です。総合スコアでは mE5-small には若干及ばないまでも、タスクによっては勝っていたりしますし、[他の日本語baseサイズbertモデルよりもスコアが高いこともある](#)ぐらい、最低限実用できそうな性能が出ていますね。本当にそんなに性能が出るのか実際に学習させてみるまでは半信半疑でしたが、驚きです。

Model	Avg(micro)	Retrieval	STS	Classification	Reranking	Clustering	PairClassification
text-embedding-3-small	69.18	66.39	79.46	73.06	92.92	51.06	62.27
multilingual-e5-small	67.71	67.27	80.07	67.62	93.03	46.91	62.19
static-embedding-japanese	67.17	67.92	80.16	67.96	91.87	40.39	62.37

なお、StaticEmbedding 日本語モデル学習などの技術的なことは記事の後半に書いているので、興味がある方はどうぞ。

利用方法

利用は簡単、SentenceTransformer を使っていつもの方法で文章ベクトルを作れます。今回はGPUを使わず、CPUで実行してみましょう。なお SentenceTransformer は 3.3.1 で試しています。

```
pip install "sentence-transformers>=3.3.1"
```

```
from sentence_transformers import SentenceTransformer

model_name = "hotchpotch/static-embedding-japanese"
model = SentenceTransformer(model_name, device="cpu")

query = "美味しいラーメン屋に行きたい"
docs = [
    "素敵なカフェが近所にあるよ。落ち着いた雰囲気できゅっくりできるし、窓際の席からは公園の景色も見えるんだ。",
    "新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕も確かです。",
    "あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。",
    "おすすめの中華そばの店を教えてあげる。とりわけチャーシューが手作りで柔らかくてジューシーなんだ。",
]

embeddings = model.encode([query] + docs)
print(embeddings.shape)
similarities = model.similarity(embeddings[0], embeddings[1:])
for i, similarity in enumerate(similarities[0].tolist()):
    print(f"{similarity:.04f}: {docs[i]}")
```

```
(5, 1024)
0.1040: 素敵なカフェが近所にあるよ。落ち着いた雰囲気できゅっくりできるし、窓際の席からは公園の景色も見えるんだ。
0.2521: 新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕も確かです。
0.4835: あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。
0.3199: おすすめの中華そばの店を教えてあげる。とりわけチャーシューが手作りで柔らかくてジューシーなんだ。
```

このように、queryにマッチする文章のスコアが高くなるように計算できてますね。この例文では、例えばBM25ではqueryに含まれる「ラーメン」のような直接的な単語が文章に出ていないため、うまくマッチさせることが難しいでしょう。

続いて、類似文章タスクの例です。

```
sentences = [
    "明日の午後から雨が降るみたいです。",
    "来週の日曜日は天気が良いそう。",
    "あしたの昼過ぎから傘が必要になりそう。",
    "週末は晴れるという予報が出ています。",
]

embeddings = model.encode(sentences)
similarities = model.similarity(embeddings, embeddings)

print(similarities)

# 一つ目の文章と、その他の文章の類似度を表示
for i, similarity in enumerate(similarities[0].tolist()):
    print(f"{similarity:.04f}: {sentences[i]}")
```

```
tensor([[1.0000, 0.2814, 0.3620, 0.2818],
        [0.2814, 1.0000, 0.2007, 0.5372],
        [0.3620, 0.2007, 1.0000, 0.1299],
        [0.2818, 0.5372, 0.1299, 1.0000]])
1.0000: 明日の午後から雨が降るみたいです。
0.2814: 来週の日曜日は天気が良いそう。
0.3620: あしたの昼過ぎから傘が必要になりそう。
0.2818: 週末は晴れるという予報が出ています。
```

こちら、類似文章が高スコアになる結果になりました。

またTransformerモデルを利用してCPUで文章ベクトルを作った場合、少ない文章量でもだいぶ時間がかか、という経験をされた方も多いと思います。StaticEmbedding モデルではCPUがそこそこ速ければ一瞬で終わるはず。さすが100倍速。

出力次元を小さくする

標準で作られる文ベクトルの次元は1024ですが、これをさらに小さく次元削減することもできます。例えば 128 を指定してみましょう。

```
# truncate_dim は 32, 64, 128, 256, 512, 1024 から指定
model = SentenceTransformer(model_name, device="cpu", truncate_dim=128)

query = "美味しいラーメン屋に行きたい"
docs = [
    "素敵なカフェが近所にあるよ。落ち着いた雰囲気できゅっくりできるし、窓際の席からは公園の景色も見えるんだ。",
    "新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕も確かです。",
    "あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。",
    "おすすめの中華そばの店を教えてあげる。とりわけチャーシューが手作りで柔らかくてジューシーなんだ。",
]

embeddings = model.encode([query] + docs)
print(embeddings.shape)
similarities = model.similarity(embeddings[0], embeddings[1:])
for i, similarity in enumerate(similarities[0].tolist()):
    print(f"{similarity:.04f}: {docs[i]}")
```

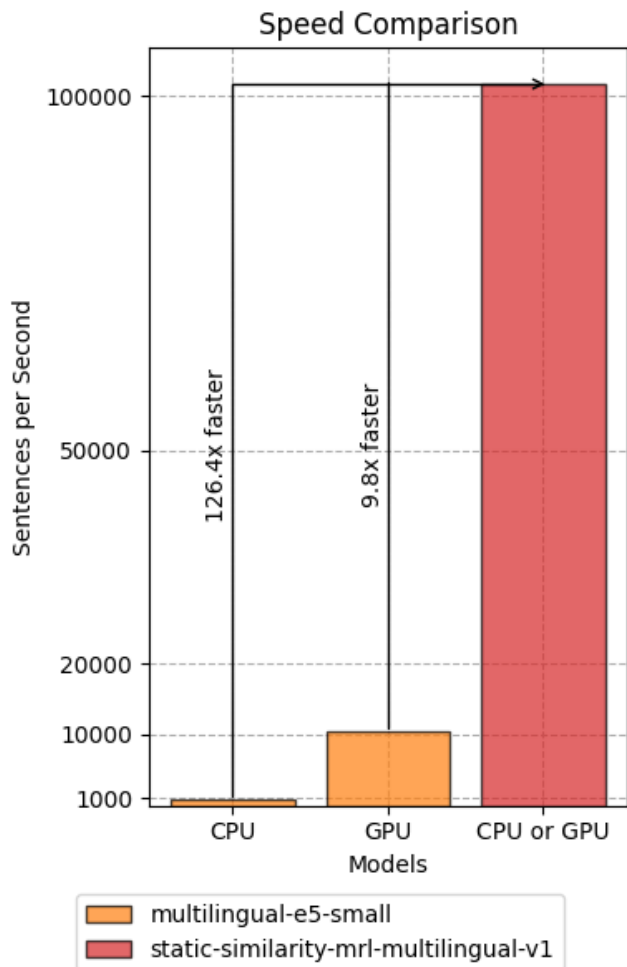
```
(5, 128)
0.1464: 素敵なカフェが近所にあるよ。落ち着いた雰囲気できゅっくりできるし、窓際の席からは公園の景色も見えるんだ。
0.3094: 新鮮な魚介を提供する店です。地元の漁師から直接仕入れているので鮮度は抜群ですし、料理人の腕も確かです。
0.5923: あそこは行きにくいけど、隠れた豚骨の名店だよ。スープが最高だし、麺の硬さも好み。
0.3405: おすすめの中華そばの店を教えてあげる。とりわけチャーシューが手作りで柔らかくてジューシーなんだ。
```

128次元のベクトルになり、結果のスコアも若干変わりましたね。次元が小さくなったことで、性能が少々劣化しています(後半にベンチマークを記載)。ただ1024次元から128次元に減ることで、保存するストレージサイズが減ったり、検索時などに利用する類似度計算コストが約8倍速になったりとなったりと、用途によっては小さい次元の方が嬉しいことも多いでしょう。

なぜCPUで推論が高速なの？

StaticEmbedding はTransformerモデルではありません。つまりTransformerの特徴である "Attention Is All You Need" なアテンションの計算が一切ないのです。文章に出てくる単語トークンを1024次元のテーブルに保存して、文ベクトル作成時にはその平均をとっているだけです。なお、アテンションがないので、文脈の理解などはしていません。

また内部実装では PyTorch の nn.EmbeddingBag を使って、全てを連結したトークンとオフセットを渡して処理することで、PyTorch の最適化で高速なCPU並列処理とメモリアクセスがされているようです。



元記事の速度評価結果によるとCPUではmE5-smallと比べて126倍速らしいですね。

評価結果

JMTEBでの全ての評価結果は[こちらJSONファイルに記載](#)しています。[JMTEB Leaderboard](#)で他のモデルと見比べると、相対的な差がわかるでしょう。JMTEBの全体の評価結果はモデルサイズを考えると、すこぶる良好です。なお、JMTEBのmr-tidyタスクは700万文章のベクトル化を行うので処理に時間がかかりかかる(モデルにもよりますがRTX4090で1~4時間ほど)と思います。これもStaticEmbeddingsでは非常に速く、RTX4090では約4分で処理終わることができました。

情報検索でBM25の置き換えができそうか?

JMTEBの中の情報検索タスクの[Retrievalの結果](#)を見てみましょう。StaticEmbeddingではmr-tidyの項目が著しく悪いですね。mr-tidyは他のタスクに比べて文章量が圧倒的に多く(700万文章)、つまり所大量の文章を検索するようなタスクでは結果が悪い可能性があります。文脈を無視した単純なトークンの平均なので、増えれば増えるほど似た平均の文章が出てくるとすると、そういう結果にもなり得そうですね。

ので、大量の文章の場合、BM25よりもだいぶ性能が悪い可能性があります。ただ、少ない文章で、ずばりの単語マッチが少ない場合は、BM25よりも良好な結果になることが多そうですね。

なお情報検索タスクの jaqket の結果が他のモデルに対してやたら良いのは、jaqket の問題を含む JQaRa (dev, unused)を学習しているからといっても、高すぎる感じで謎です。test の情報リークはしていないと思うのですが...。

クラスタリング結果が悪い

こちらも詳細は追っかけていませんが、スコア的には他のモデルよりもだいぶ悪い結果ですね。クラス分類タスクは悪くないので不思議です。埋め込み空間がマトリョーシカ表現学習で作られた影響もあるのでしょうか。

JQaRA, JaCWIR でのリランキングタスク評価

JQaRA の結果はこちら。

model_names	ndcg@10	mrr@10
static-embedding-japanese	0.4704	0.6814
bm25	0.458	0.702
multilingual-e5-small	0.4917	0.7291

JaCWIR の結果はこちら。

model_names	map@10	hits@10
static-embedding-japanese	0.7642	0.9266
bm25	0.8408	0.9528
multilingual-e5-small	0.869	0.97

JQaRa 評価は BM25 より若干良く、mE5-small より若干低い、JaCWIR は BM25, mE5 よりだいぶ低い感じの結果になりました。

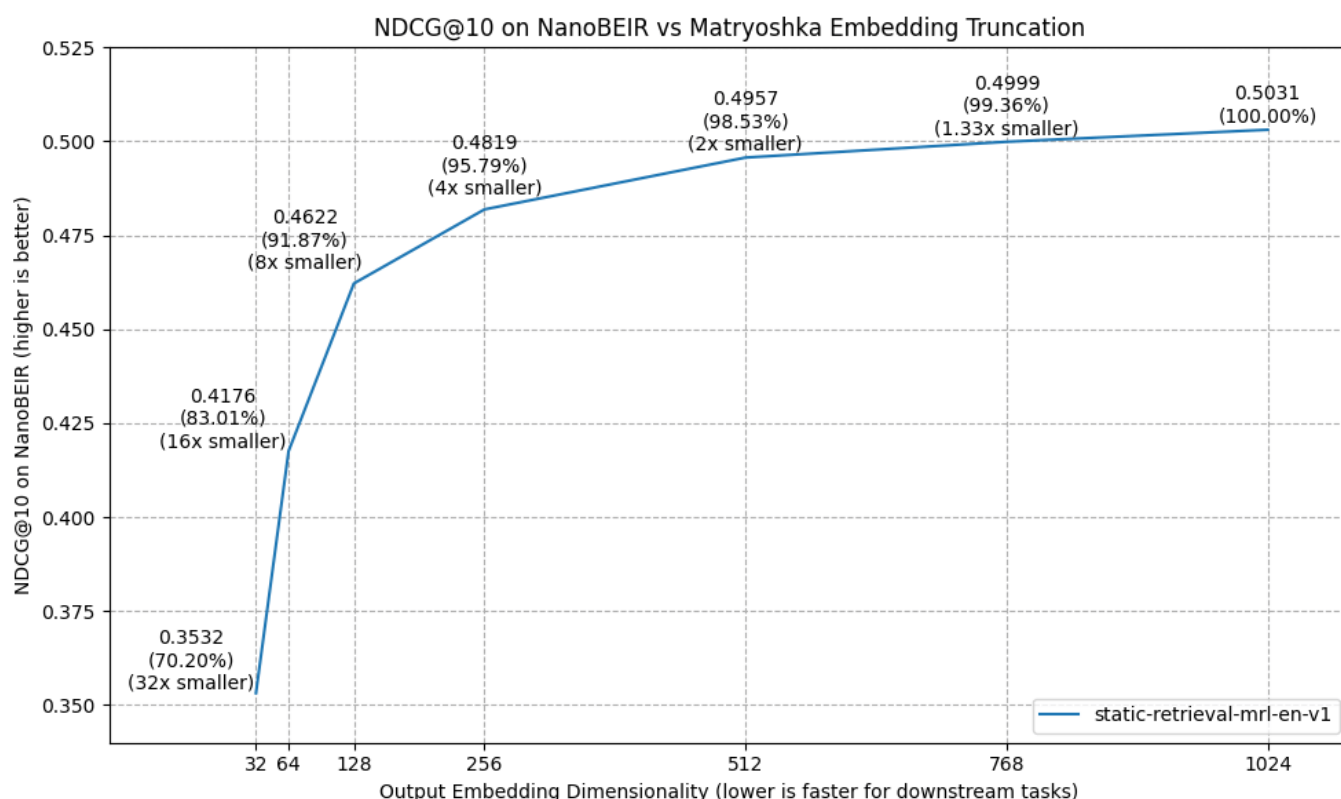
JaCWIR はqueryから探しあてる文章が、Web文章のタイトルと概要文なので、いわゆる「綺麗な」文章ではないケースも多いです。transformerモデルはノイズに強いので、単純なトークン平均のStaticEmbeddingではスコアに差がつけられるのも納得ですね。BM25は特徴的な単語が出現した文章にマッチするので、JaCWIR でもノイズとなるような文章上の単語はクエリにそもそもマッチしないため、Transformer モデルと競争力のある結構良い結果を残しています。

この結果から、StaticEmbedding は Transformer / BM25 に比べ、ノイズを多く含む文章の場合はスコアが悪い可能性があります。

出力次元の削減

StaticEmbedding で出力される次元は、学習次第ですが今回作成したものは1024次元とそこそこのサイズです。次元数が大きいと、推論後のタスク(クラスタリングや情報検索など)に計算コストがかかってしまいます。しかしながら、学習時にマトリョーシカ表現学習([Matryoshka Representation Learning\(MRL\)](#))をしているため、1024次元をさらに小さな次元へと簡単に次元削減ができます。

MRLは、学習時に先頭のベクトルほど重要な次元を持つてくることで、例えば1024次元でも先頭の32,64,128,256...次元だけを使って後ろを切り捨てるだけで、ある程度良好な結果を示しています。



このグラフ参照元の[StaticEmbedding の記事](#)によると、128次元で91.87%, 256次元で95.79%, 512次元で98.53%の性能を維持しているようです。精度にそこまでシビアではないが、その後の計算コストを下げたい場合、ガッと次元削減して使う、という用途にも使えそうですね。

StaticEmbdding 日本語モデルでの次元削減結果

JMTEB では、出力時にモデルのパラメータを制御できるため、truncate_dim オプションを渡すことで、次元削減した結果のベンチマークも簡単に計測できます。素晴らしいですね。というわけで、StaticEmbdding 日本語モデルで

も、次元削減した結果でベンチマークをとってみました。

次元数	Avg(micro)	スコア割合 (%)	Retrieval	STS	Classification	Reranking	Clustering	PairClassification
1024	67.17	100.00	67.92	80.16	67.96	91.87	40.39	62.37
512	66.57	99.10	67.63	80.11	65.66	91.54	41.25	62.37
256	65.94	98.17	66.99	79.93	63.53	91.73	42.55	62.37
128	64.25	95.65	64.87	79.56	60.52	91.62	41.81	62.33
64	61.79	91.98	61.15	78.34	58.23	91.50	39.11	62.35
32	57.93	86.24	53.35	76.51	55.95	91.15	38.20	62.37

スコアの変化を見ると、512次元へと次元削減した場合はやたらRetrieval, Classification, Reranking の性能が悪くなります。むしろ256次元まで次元削減してしまった方が良好な結果に。256次元では、スコア的には次元削減する前のモデルの98.93%なのですが、これはクラスタリングの結果がなぜか1024次元よりも良くなってしまったためですね。

512次元でのスコア計測が間違っていたので修正しました。マトリョーシカ表現学習がうまく反映され、次元数を削ると若干のスコア低下が見られますが、次元数が減ったためその後のコストが抑えられそうですね。

クラスタリングタスクにおいては128次元まで次元削減しても1024次元よりもスコアが高い、という本来情報量を削らない方がスコアが良くなりそうなのに、クラスタリングタスクのみは逆にスコアが上がってしまう興味深い結果となりました…。マトリョーシカ表現学習では、先頭の次元の方が全体的な特徴を踏まえているので、クラスタリング用途には(クラスタリングのアルゴリズムにもよりますが)、特徴的な前の方の次元のみで後ろの次元を使わない方が良質な結果が得られる、ということなのかもしれません。

というわけで、static-embedding-japanese モデルで次元削減する時は、512,256,128次元あたりが性能と次元削減のバランスが取れてそうですね。

StaticEmbedding モデルを作ってみて

正直、単純なトークンのembeddingsの平均でそんなに性能出なのか半信半疑だったのですが、実際に学習させてみてシンプルなアーキテクチャなのに性能の高さにびっくりしました。Transformer 全盛のこの時代に、古き良き単語埋め込みの活用モデルで、実世界で活用できそうなモデルの出現に驚きを隠せません。

CPUでの推論速度が速い文ベクトル作成モデルは、ローカルCPU環境で大量の文章の変換などはもとより、エッジデバイスだったりネットワークが遅い(リモートの推論サーバを叩けない)環境だったり、色々と活用できそうですね。



StaticEmbedding 日本語モデル学習のテクニカルノート

なぜうまく学習できるのか

StaticEmbedding は非常にシンプルで、文章をトークナイズしたIDで単語の埋め込みベクトルが格納されている EmbeddingBag テーブルからN次元(今回は1024次元)のベクトルを取得し、その平均を取るだけです。

これまで、単語埋め込みベクトルといえば、word2vec や GloVe のように Skip-gram や CBOW を用いて単語の周辺を学習してきました。しかし、StaticEmbedding では文章全体を用いて学習しています。また、対照学習を使って大量の様々な文章を巨大バッチで学習しており、良い単語の埋め込み表現の学習に成功しています。

対照学習は、基本的に正例以外全てを負例として学習するため、例えばバッチサイズ2048なら1の正例に対して2047の負例を2048通り、つまり2048x2047で約400万の比較を学習します。そのため、元の単語空間に対して適切な重みを更新しながら、学習を進めることができます。

学習データセット

日本語モデル学習にあたり、対照学習で利用できるデータセットとして、以下を作成し使用しました。

- [hotchpotch/sentence_transformer_japanese](#)
 - [SentenceTransformer](#) で学習しやすいカラム名と構造に整えたものです。
 - `(anchor, positive)`, `(anchor, positive, negative)`, `(anchor, positive, negative_1, ..., negative_n)` といった構造になっています。
 - 以下のデータセットを基に [hotchpotch/sentence_transformer_japanese](#) を作成しました。毎度ながらデータセットの作者の方々・とりわけ [hpprc](#) 氏に感謝です。
 - <https://huggingface.co/datasets/hpprc/emb>
 - https://huggingface.co/datasets/hotchpotch/hpprc_emb-scores のリランカースコアを使用し、`positive(>=0.7)` / `negative(<=0.3)` のフィルタリングを行いました。
 - <https://huggingface.co/datasets/hpprc/llmjp-kaken>
 - <https://huggingface.co/datasets/hpprc/msmarco-ja>
 - <https://huggingface.co/datasets/hotchpotch/msmarco-ja-hard-negatives> のリランカースコアを用いて、`positive(>=0.7)` / `negative(<=0.3)` のフィルタリングを行いました。
 - <https://huggingface.co/datasets/hpprc/mqa-ja>
 - <https://huggingface.co/datasets/hpprc/llmjp-warp-html>
 - 上記の作成したデータセットの中で、以下を使用しました。なお、情報検索を強化したかったため、情報検索に適したデータセットのデータはオーギュメンテーションで件数を多めに学習させています。

- httprc_auto-wiki-nli-triplet
- httprc_auto-wiki-qa
- httprc_auto-wiki-qa-nemotron
- httprc_auto-wiki-qa-pair
- httprc_baobab-wiki-retrieval
- httprc_janli-triplet
- httprc_jaquad
- httprc_jqara
- httprc_jsnli-triplet
- httprc_jsquad
- httprc_miracl
- httprc_mkqa
- httprc_mkqa-triplet
- httprc_mr-tydi
- httprc_nu-mnli-triplet
- httprc_nu-snli-triplet
- httprc_quiz-no-mori
- httprc_quiz-works
- httprc_snow-triplet
- httprc_llmjp-kaken
- httprc_llmjp_warp_html
- httprc_mqa_ja
- httprc_msmarco_ja
- 英語データセットには、以下のデータセットを利用しています。
 - [sentence-transformers/msmarco-co-condenser-margin-mse-sym-mnrl-mean-v1](#)
 - [sentence-transformers/squad](#)
 - [sentence-transformers/all-nli](#)
 - [sentence-transformers/trivia-qa](#)
 - [nthakur/swim-ir-monolingual](#)
 - [sentence-transformers/miracl](#)
 - [sentence-transformers/mr-tydi](#)

日本語トークナイザ

StaticEmbedding を学習するためには、HuggingFace のトークナイザライブラリの tokenizer.json 形式で処理可能なトークナイザを使うと簡単そうだったので、[hotchpotch/xlm-roberta-japanese-tokenizer](#) というトークナイザを作成しました。語彙数は 32,768 です。

このトークナイザは、wikipedia 日本語~~、wikipedia 英語(サンプリング)、cc-100(日本語, サンプリング)~~(訂正: 作成コードを確認したところ、wikipedia日本語のみを利用していました)のデータを unidic で分割し、

sentencepiece unigram で学習したものです。XLM-Roberta 形式の日本語トークナイザとしても機能します。今回はこのトークナイザを利用しました。

ハイパーパラメータ

[大元の学習コード](#)との変更点やメモは以下の通りです。

- batch_size を大元の 2048 から 6072 に設定しました。
 - 対照学習で巨大なバッチを処理するとき、同一バッチ内にポジティブとネガティブが含まれると学習に悪影響を与える可能性があります。これを防ぐために `BatchSamplers.NO_DUPLICATES` オプションがあります。しかし、バッチサイズが巨大だと同一バッチに含めないためのサンプリング処理に時間がかかることがあります。
 - 今回は `BatchSamplers.NO_DUPLICATES` を指定し、RTX4090 の 24GB に収まる 6072 に設定しました。バッチサイズはさらに大きい方が結果が良い可能性があります。
- epoch数を1から2に変更しました
 - 1よりも2の方が良い結果になりました。ただし、データサイズがもっと大きければ、1の方が良い可能性があります。
- スケジューラ
 - 標準のlinearから、経験則でより良いと感じるcosineに変更しました。
- オプティマイザ
 - 標準のAdamW のままです。adafactorに変更した場合、収束が悪くなりました。
- learning_rate
 - 2e-1 のままです。値が巨大すぎるのではないかと疑問に思いましたが、低くすると結果が悪化しました。
- dataloader_prefetch_factor=4
- dataloader_num_workers=15
 - トークナイズとバッチサンブラのサンプリングに時間がかかるため、大きめに設定しました。

学習リソース

- CPU
 - Ryzen9 7950X
- GPU
 - RTX4090
- memory
 - 64GB

このマシンリソースで、フルスクラッチ学習にかかった時間は約4時間でした。GPUのコア負荷は非常に小さく、他のtransformerモデルでは学習時に90%前後で張り付くのに対して、StaticEmbeddingではほとんど0%でした。これは、巨大なバッチをGPUメモリに転送する時間が大半を占めているためかと思われます。そのため、GPUメモリの帯域幅が速くなれば、学習速度がさらに向上する可能性があります。

さらなる性能向上へ

今回利用したトークナイザはStaticEmbedding向けに特化したものではないため、より適したトークナイザを使用すれば性能が向上する可能性があります。バッチサイズをさらに巨大化することで、学習の安定性が向上し、性能向上が見込めるかもしれません。

また、さまざまなドメインや合成データセットを利用するなど、より幅広い文章リソースを学習に組み込むことで、さらなる性能向上が期待できます。

大元の学習コード

学習に使用したコードは、以下で MIT ライセンスで公開しています。スクリプトを実行すれば再現できる、はず...!

- <https://huggingface.co/hotchpotch/static-embedding-japanese/blob/main/trainer.py>

ライセンス

static-embedding-japanese はモデル重み・学習コードを MIT ライセンスで公開しています。

[記事の一覧 >](#)

関連するかもエントリー

情報検索モデルで最高性能(512トークン以下)・日本語版SPLADE v2をリリース / 2024年12月19日

2024年は情報検索技術に興味を持ち、情報検索関連のモデル作りを趣味で行っている @hotchpotch (セコン)です。Transformer は割と適当にやっても、いい感じに学習してくれるので、楽しいですね。というわけで、日々部屋でご家庭用GPUを回し、以前公開した情報検索に特...

OpenAIの新embeddings,text-embedding-3-smallをRAGタスクで評価する / 2024年1月29日

先日、OpenAI から新しい embeddings モデルである、text-embedding-3-smallとtext-embedding-3-largeが公開された。text-embedding-3-smallは、古いembeddingsモデルのada-v2よりも価格は1/...

SPLADE モデルの作り方・日本語SPLADEテクニカルレポート / 2024年10月22日

近年、大規模言語モデル(LLM)の台頭により、情報検索の重要性が増している。特に、Retrieval-Augmented Generation(RAG)などの応用分野では、効率的で高精度な検索システムが求められている。ニューラルネットワークを用いた検索モデルの分野では、密ベクトル...